CHAPTER 1

Linked Lists and C++ transition

1.1 From C to C++

In this chapter we are going to transition from stand C programming to C++. Why C++? Because it is a more modern version of C++ that allows for a deeper insight into the structures that we put into the memory of the computer. C++ differs in that it allows for object-oriented programming, which is a programming paradigm that uses "objects" to represent data and methods. This allows for more complex data structures and algorithms to be implemented in a more organized and efficient manner. To do this, we will be utilizing *classes*, which we briefly covered in our Advanced Python chapter. In C++, classes are defined using the `struct` keyword.

There are two additional keyword differences you should be aware of regarding C vs C++. The primary one is the usage of the keyword *new*, which does not exist in C. In C, we have `malloc()`, which frees up a given amount of memory in bytes, and `sizeof`, which returns the size in bytes of a data structure.

```
1    // C : what you already know
2    struct Car *n = malloc(sizeof(struct Car));
```

C++ combines these two commands into one, called `new`.

```
1    // C++ : same thing, no sizeof, no cast
2    Car* n = new Car();
```

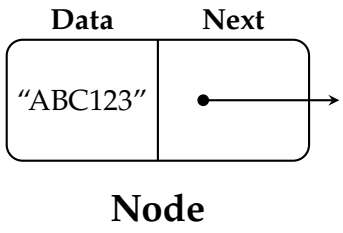
Here is a helpful video on some of the deeper differences that we will not be covering: https://www.youtube.com/watch?v=B_4x7WlQr7M

1.2 Linked Lists

A *linked list* is a linear data structure where each element is a separate object, called a *node*. Each node holds its own data and the address of the next node, called a *successor*, thus forming a chain-like structure. The first node in the list is called the *head*. The head is the only node you need to keep a direct reference to, since every other node can be reached by following successors from it. The last node, the one whose successor pointer is empty, is called the *tail*.

A simple node in a linked list can be represented in C++ as follows:

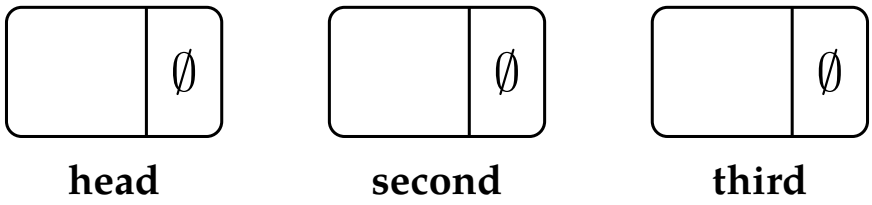
```
struct Node {
    int data;
    Node* next;
};
```



In this structure, ’data’ is used to store the data and ’next’ is a pointer that holds the address of the next Node in the list. When a pointer has nothing to point to, we mark its field with the empty-set symbol $\emptyset$ — the mathematical symbol for “nothing here.”

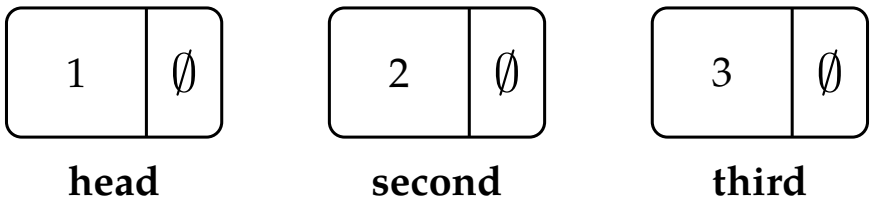
Here is a simple example of creating and linking nodes in a linked list:

```
// Create nodes
Node* head = new Node();
Node* second = new Node();
Node* third = new Node();
```

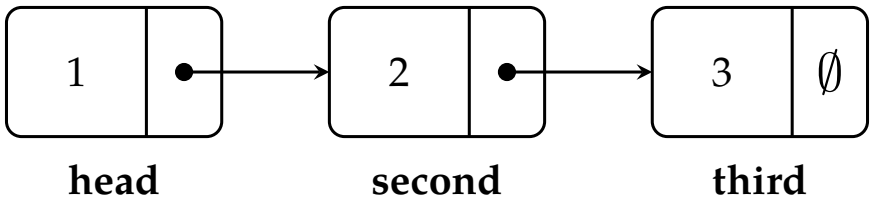



```

// Assign data
head->data = 1;
second->data = 2;
third->data = 3;
```




```
// Link nodes
head->next = second;
second->next = third;
third->next = nullptr;  // The last node points to null, this is the TAIL
```



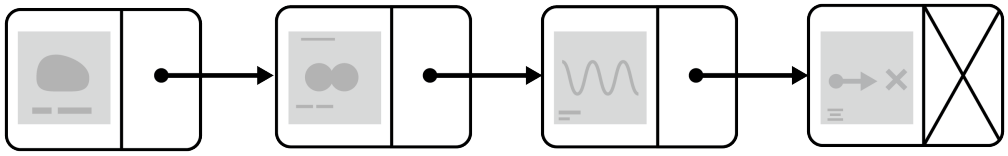
In this example, we first create three nodes using the ‘new‘ keyword, which dynamically allocates memory. We then assign data to the nodes and link them using the ’next’ pointer.

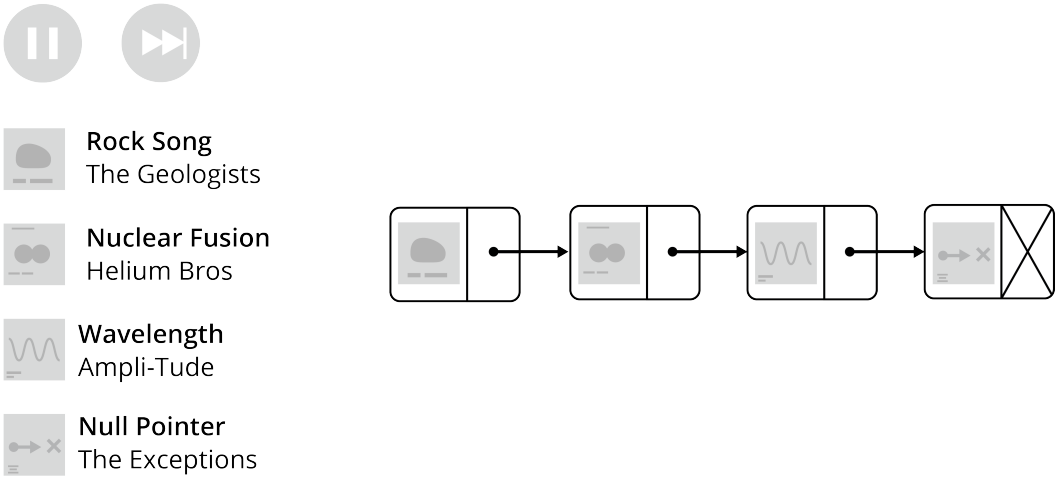
Rock Song
The Geologists

Nuclear Fusion
Helium Bros

Wavelength
Ampli-Tude

Null Pointer
The Exceptions

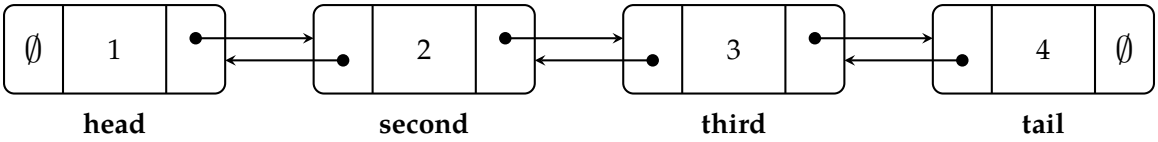



Rock Song
The Geologists
Nuclear Fusion
Helium Bros
Wavelength
Ampli-Tude
Null Pointer
The Exceptions

1.3 Doubly Linked Lists

Now that we have established the basics of linked lists, we can move on to a more complex structure: the *doubly linked list*. Doubly linked lists are strong because each node now has two pointers: the same next pointer, and now, the *predecessor*, which we will call `prev`.

```
struct DoublyNode {
    int data;
    DoublyNode* next;
    DoublyNode* prev;
};
```



Given this structure, we can traverse the entire list starting at the tail to the head or from the head to the tail.

Let's construct some code to analyze where the nodes would be in memory. We will create a doubly linked list with four nodes, and then we will print out the addresses of each node and their respective next and previous pointers.

Write the following code for the doubly linked list. There is a full file in your digital resources, here is a snippet of the code to get you started.

```

int main() {
    // Create nodes
    DoublyNode* head = new DoublyNode();
    DoublyNode* second = new DoublyNode();
    DoublyNode* third = new DoublyNode();
    DoublyNode* tail = new DoublyNode();

    // Assign data
    head->data = 1;
    second->data = 2;
    third->data = 3;
    tail->data = 4;

    // Link nodes forward
    head->next = second;
    second->next = third;
    third->next = tail;
```

```
    tail->next = nullptr;   // tail has no successor

    // Link nodes backward
    head->prev = nullptr;   // head has no predecessor
    second->prev = head;
    third->prev = second;
    tail->prev = third;

    // Print the address of each node, and the addresses stored in its
    // prev/next fields, so you can see both directions of the chain.
    std::cout << "head   is at: " << head   << "   prev: " << head->prev   << "
    ↪  (nullptr)" << "   next: " << head->next   << std::endl;
    std::cout << "second is at: " << second << "   prev: " << second->prev << "
    ↪  next: " << second->next << std::endl;
    std::cout << "third  is at: " << third  << "   prev: " << third->prev  << "
    ↪  next: " << third->next  << std::endl;
    std::cout << "tail   is at: " << tail   << "   prev: " << tail->prev   << "
    ↪  next: " << tail->next   << " (nullptr)" << std::endl;

    // Each DoublyNode takes up a fixed number of bytes in memory.
    std::cout << "\nsizeof(DoublyNode) = " << sizeof(DoublyNode) << " bytes" <<
    ↪  std::endl;

    // Clean up the memory
    delete head;
    delete second;
    delete third;
    delete tail;

    return 0;
}
```

When you run this, you'll get real addresses like these (yours specific addresses will differ!):

```
head   is at: 0x100c9dca0   prev: 0 (nullptr)   next: 0x100c9dbc0
second is at: 0x100c9dbc0   prev: 0x100c9dca0   next: 0x100c9dbe0
third  is at: 0x100c9dbe0   prev: 0x100c9dbc0   next: 0x100c9dc00
tail   is at: 0x100c9dc00   prev: 0x100c9dbe0   next: 0 (nullptr)

sizeof(DoublyNode) = 24 bytes
```

Computer memory is comprised of a very long string of bytes, and each byte has its own address. This line is a large, contiguous stream of ones and zeros when you get to the core of it. Often, programmers separate it into chunks of certain sizes so that we can make them look similar to grids of memory, like matrices.

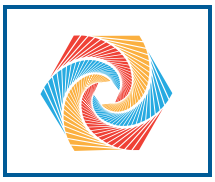
Look closely at `second` and `third`: `0x100c9dbe0 - 0x100c9dbc0 = 0x20`, which is **32** in

decimal, not 24 bytes like `sizeof(DoublyNode)` had provided. The same is true between `third` and `tail`. Why, then, is the a difference between the 32 stored bytes and 24 bytes we would expect our memory to take up?

`DoublyNode` holds an `int` (4 bytes) plus two pointers (8 bytes each on a 64-bit machine), for 24 bytes total. But heap allocators don't hand out memory in exactly the size you ask for, they *round each allocation up* to a boundary of 16 bytes. Since 24 isn't a multiple of 16, the allocator rounds it up to the next one: 32. Your struct only uses 24 of those bytes; the other 8 are allocator *padding* that `sizeof()` never shows you. This is why the command new combines `malloc` and `sizeof`, and then proceeds to calculate the next multiple of 16 bytes to allocate.

Appendix A

Answers to Exercises



Index